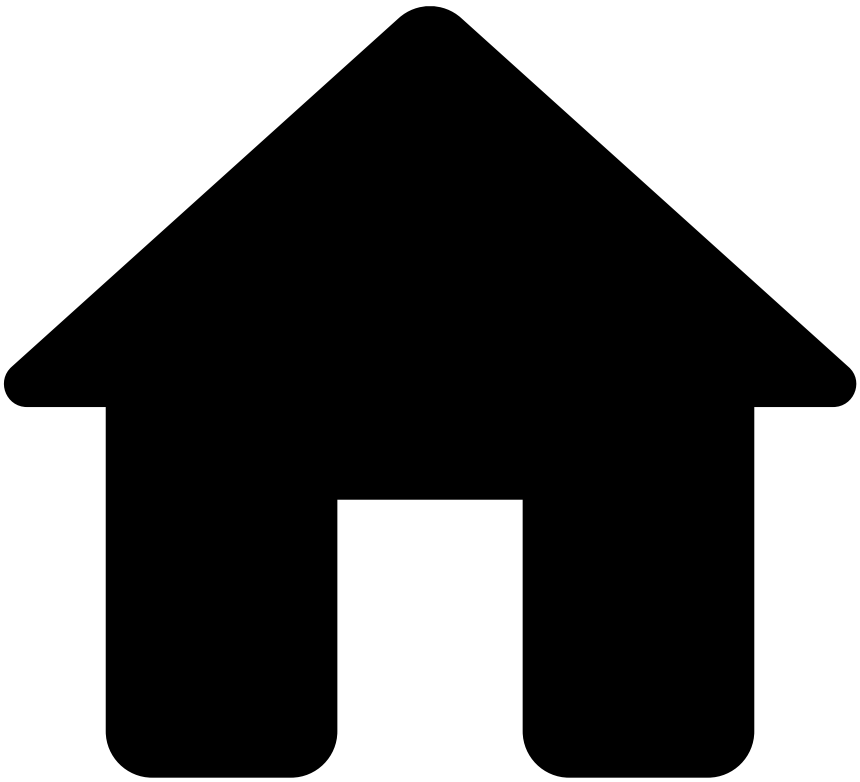
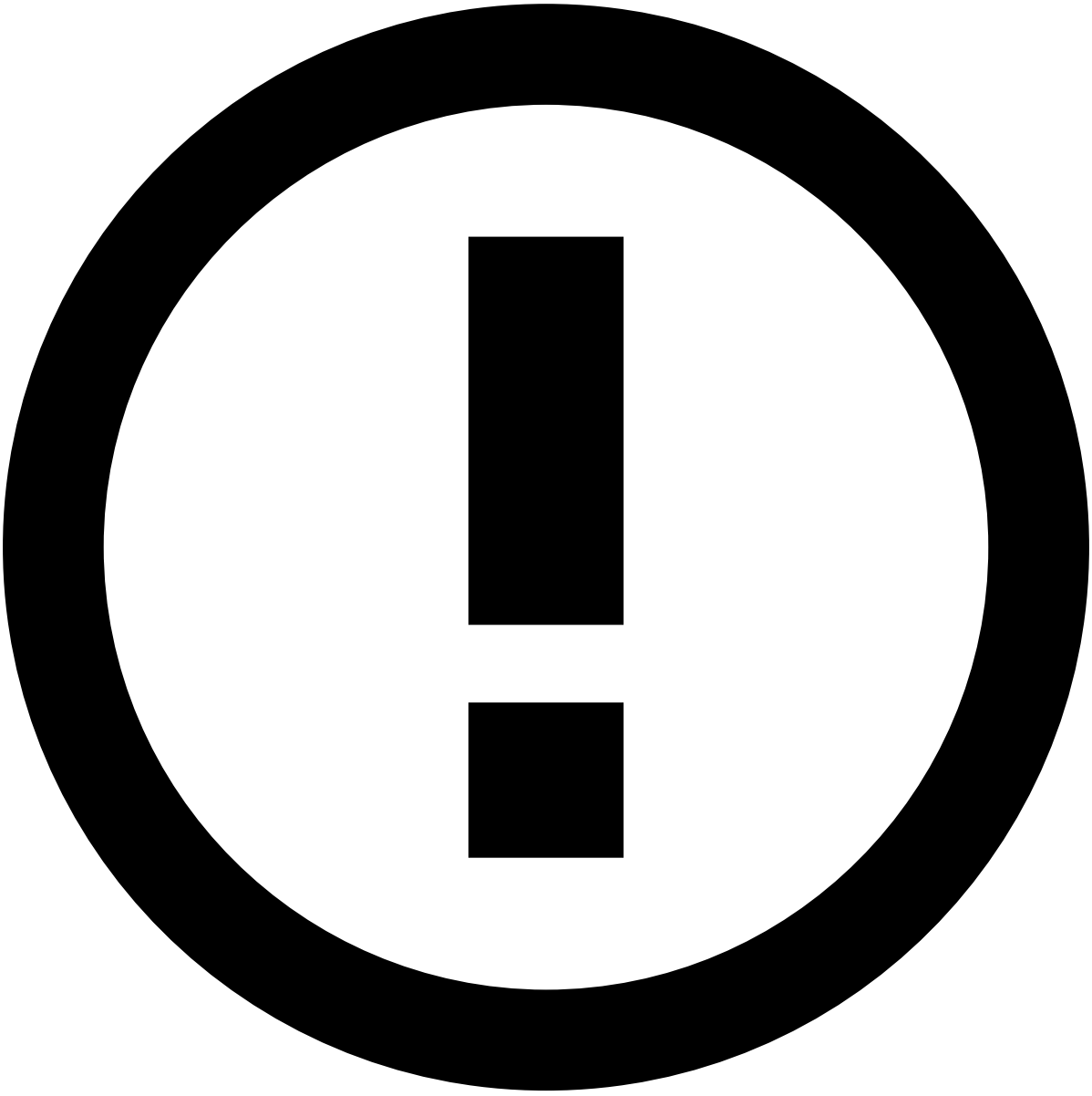


Info Endpoint

The Info endpoint enables you to update and add new information to an existing transfer, as well as to notify Feedzai about transactions processed upstream. The following section explain the life cycle for each info subtype.



info

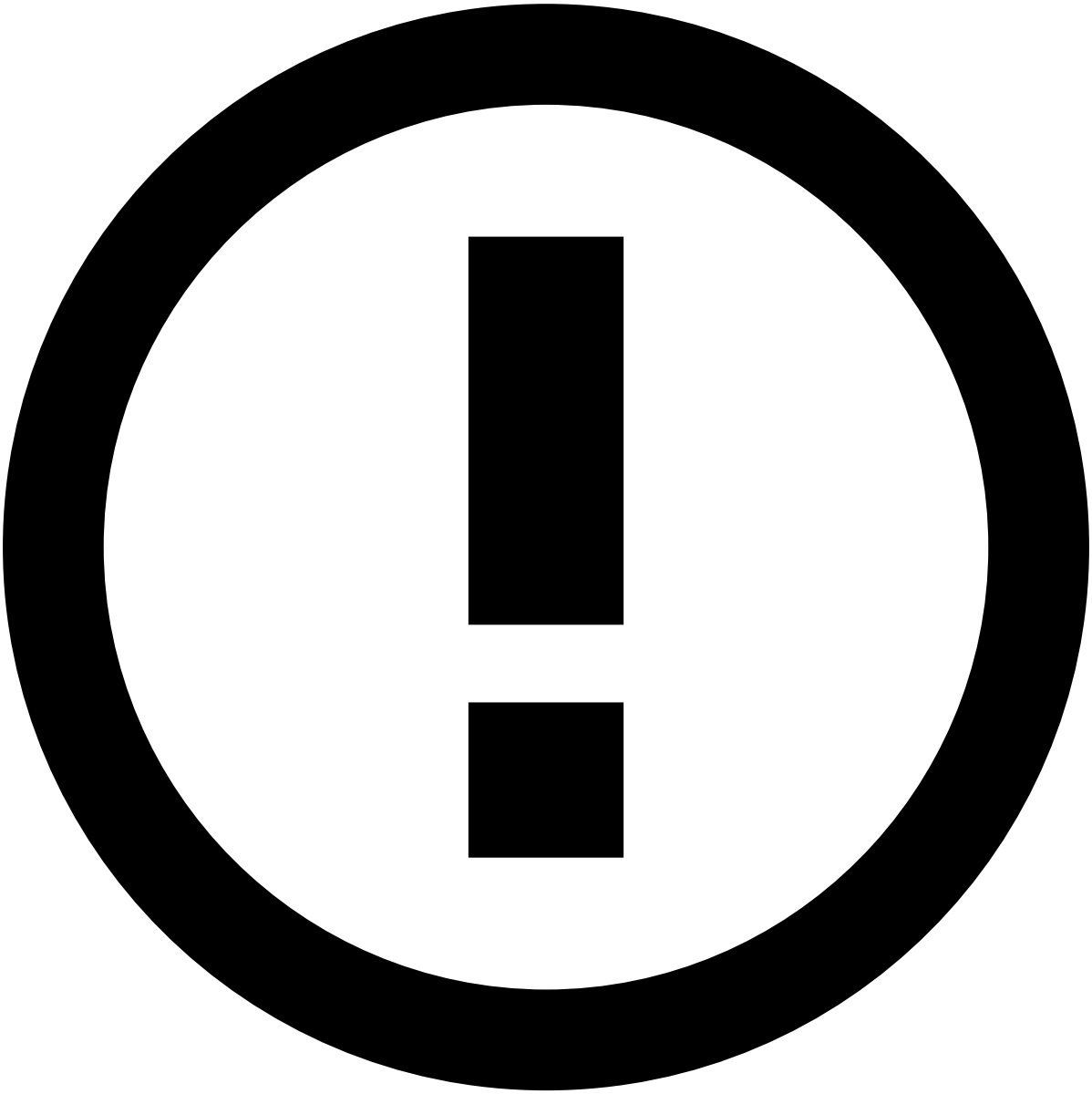
When you use the Info endpoint to perform an update, you only need to provide the new information (i.e. the original information does not need to be sent).

Additional informationâ□□

Additional Information is used to update information sent in a previous request. The following diagram shows the end-to-end process:

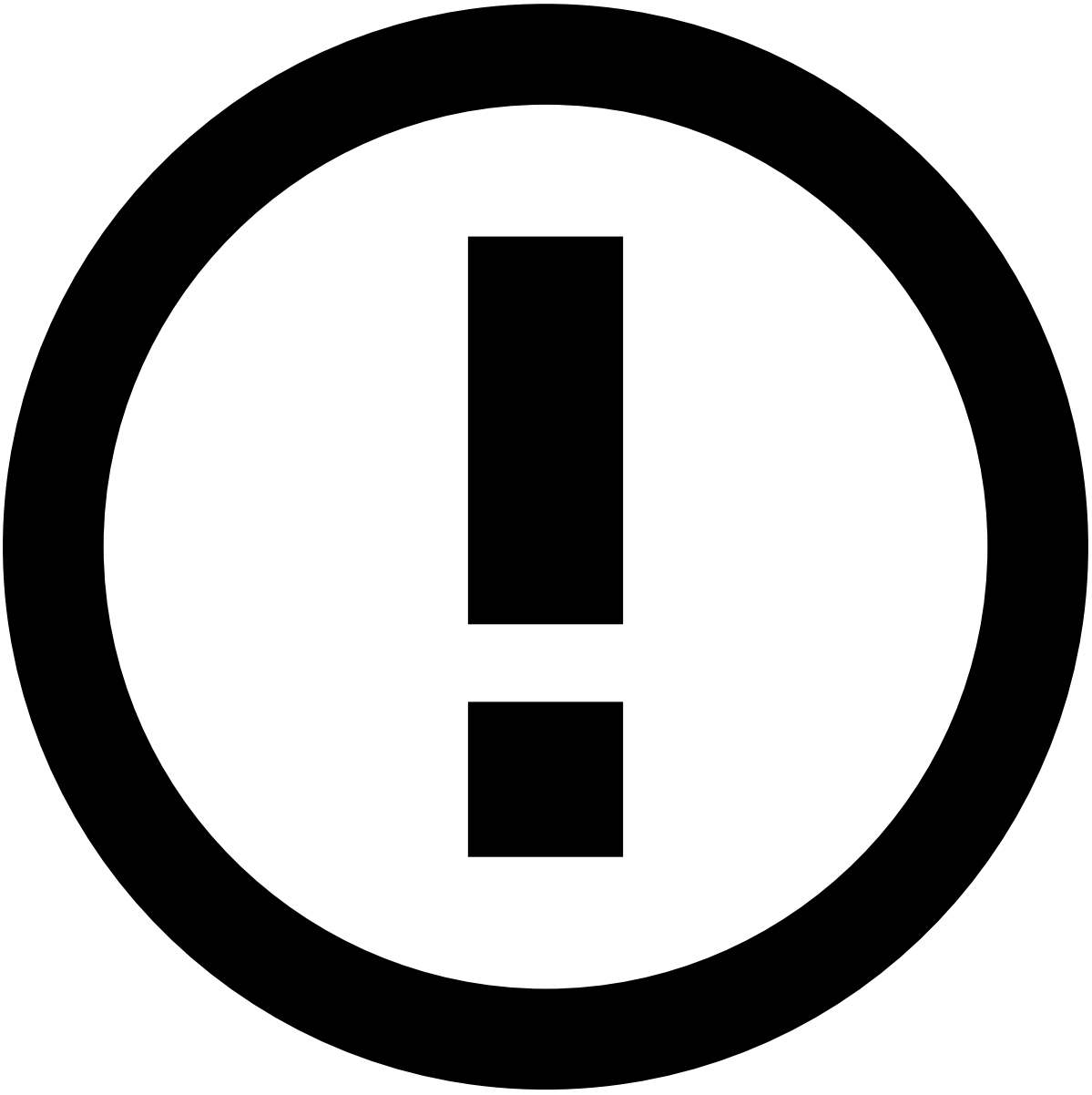
The steps depicted in the previous image are the following:

1. The sender initiates a transfer directly with the sender bank.
2. The sender bank sends an API request via the **Transfer Initiation** endpoint.
3. The sender bank sends the transfer instructions to the receiver bank.
4. The receiver's account is updated with the funds.
5. The sender bank sends an API request via the **Info** endpoint with subtype `additional_info` to update or add new information.



reference data enrichment

The event may become enriched with data fetched from the Reference Data Storage if any of the event's entities (e.g. Card, Customer, or Account) have a match in the Reference Data Storage.



info

Note that any new data from transfer info events (e.g. new `sender_card_category`) will take precedence over what's in the Reference Data Storage. Read the Reference Data Life Cycle page to understand the logic behind reference data's entity-based enrichment.

Pre-decidedâ□□

Pre-decided is used with transfers that have been either rejected or accepted before arriving at Feedzai (i.e. does not have the Feedzai recommendation or score). Typical transfers use cases include:

- Rejection at the network level before it arrives at the receiver.
- Rejection at the sender side, for example, for lack of account balance or blocked account.
- Internal transfer that is automatically accepted.
- Initiated and immediately canceled by the sender.

The following diagram shows the end-to-end process:

The steps depicted in the previous image are the following:

1. The sender initiates a transfer directly with the sender bank. If the transfer is accepted by the sender bank, then the following steps 2 and 3 take place.
2. The sender bank sends the transfer instructions to the receiver bank.
3. The receiver's account is updated with the funds.
4. The sender bank sends an API request via the **Info** endpoint with subtype `pre_decided` to inform Feedzai about the existing transfer with the final decision if the transactions were accepted or rejected.

Not-honorâ□□

Not-honor is used when you take a different decision than the one recommended by Feedzai:

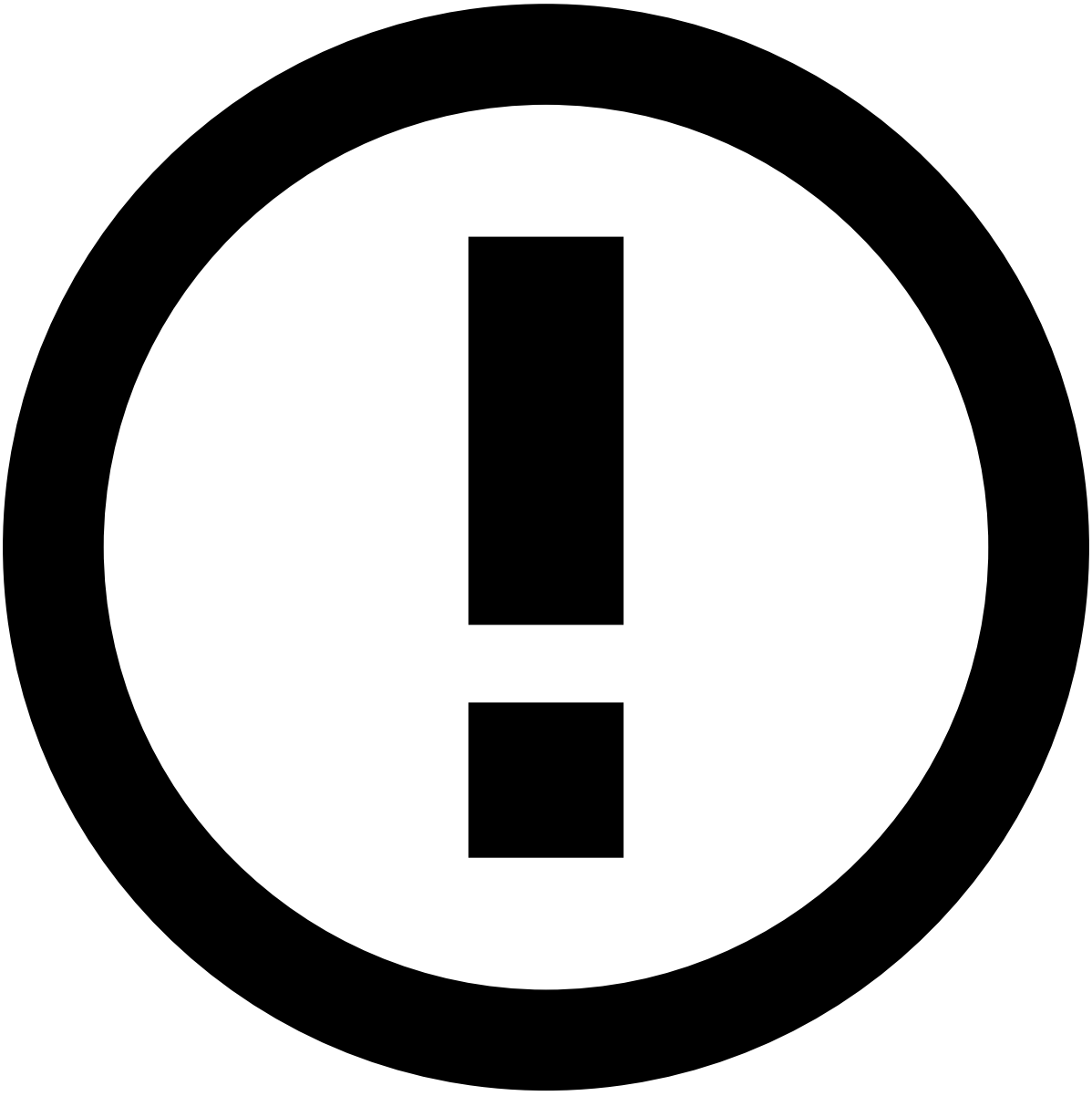
- Feedzai generates an 'approve' recommendation, and you decide to reject the transfer.
- Feedzai generates a 'decline' recommendation, and you decide to accept the transfer.

The following diagram shows the end-to-end process for the two examples described above:

The steps depicted in the previous image for the **first scenario** are the following:

1. The sender initiates a transfer directly with the sender bank.
2. The sender bank sends an API request via the **Transfer Initiation** endpoint. Feedzai generates a recommendation to accept the transfer.
3. The sender bank rejects the transaction and sends the information to the sender.
4. The sender bank sends an API request via the **Info** endpoint with subtype `not_honor` to inform Feedzai about the transfer's final decision.

In the second scenario, the transfer is accepted by the sender bank and follows the normal path towards updating the receiver account. At the end, sender bank sends an API request via the **Info** endpoint with subtype `not_honor` to inform Feedzai about the transfer's acceptance.



info

If the transfer is not originally sent to Feedzai, then pre-decided subtype needs to be used instead of the not-honor to communicate the transfer information to Feedzai.

Out of bandâ□□

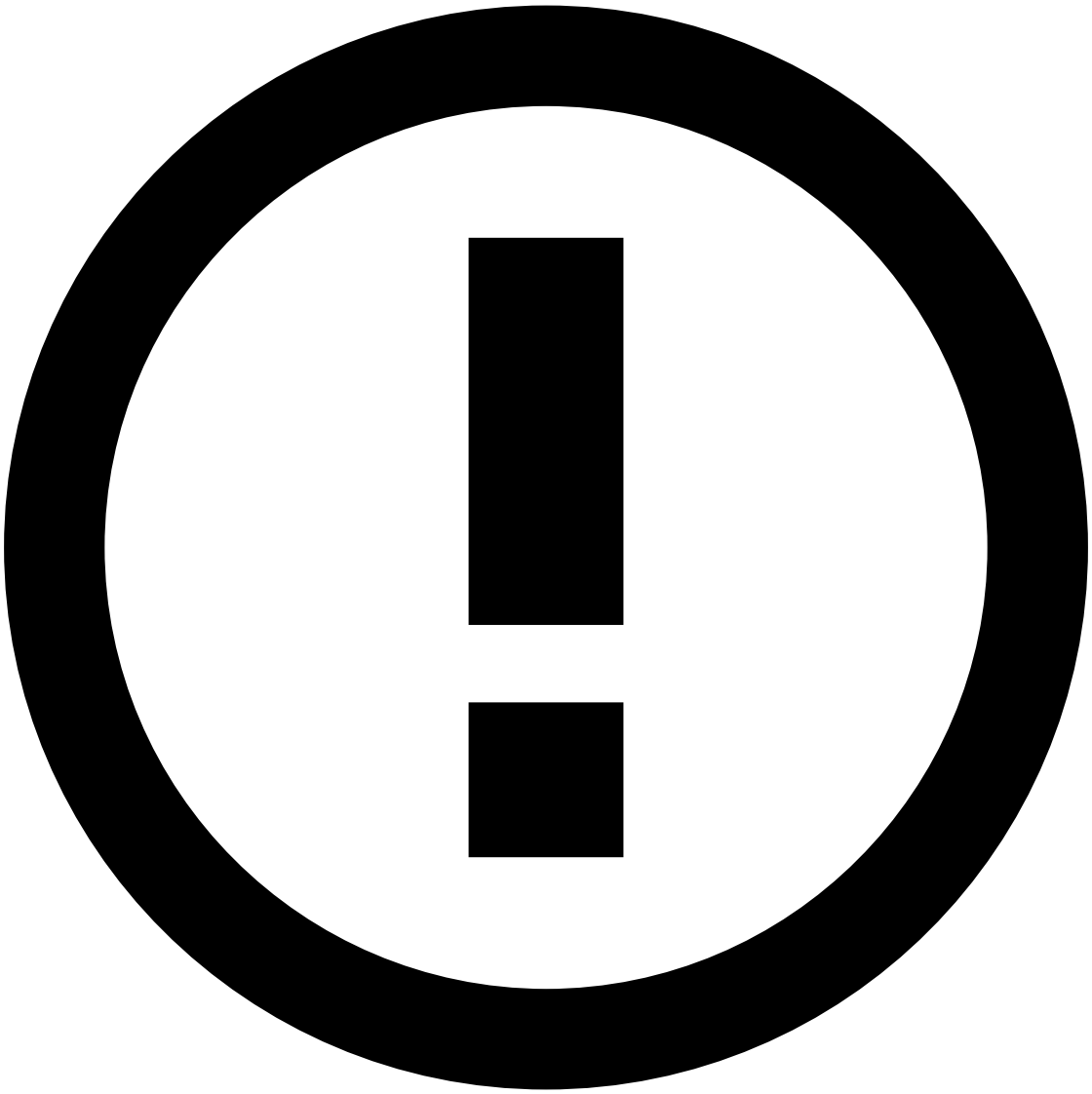
You can define policies for advising downstream systems to contact an account holder (e.g. via sms or call) to validate a transfer. These policies can be defined, for example, based on the response provided by the Feedzai's model score or rules, another output parameter (e.g. an action triggered by a rule), or any external reason. Out of band allows you to notify Feedzai that the account holder has been contacted. If the response provided by the account holder leads to a decision about whether or not the transfer is legitimate, then this information is sent via the Feedback use case.

Consider the following example of a typical scenario: An account holder can be contacted and not answer synchronously. In this situation, this information is collected via info, subtype `out_of_band`. Once the bank has the account holder's confirmation, the information is sent via the Feedback endpoint.

The following diagram shows the end-to-end process:

The steps depicted in the previous image for the API integration at the **sender bank (top figure)** are the following:

1. The sender sends the transfer instructions to the sender bank.
2. The sender bank sends an API request via the **Transfer Initiation** endpoint. Feedzai generates a recommendation to decline the transfer.
3. The sender bank contacts the account holder to verify the transfer authenticity.
4. The account holder checks the information provided by the bank.



info

Transfer's confirmation: If the account holder confirms the transfer, the information is sent via the Feedback endpoint. Based on the answer, the transfer may or may not follow the normal path towards updating the receiver account.

5. The sender bank sends an API request via the **Info** endpoint with subtype `out_of_band` to inform Feedzai that the account holder has been contacted.

As for the API integration at the receiver bank, the steps follow the same logic as the sender bank integration; however, the receiver bank contacts the receiver to verify the transfer authenticity. After that, the receiver bank sends the API call to inform Feedzai that the account holder has been contacted.

Now consider a second scenario in which the integration is done via the sender bank and the transfer is rejected by the receiver bank:

In this situation, the logic of steps is the same as the previous scenario and you can use the **Info** endpoint with subtype `out_of_band` to inform that the account holder has been contacted.

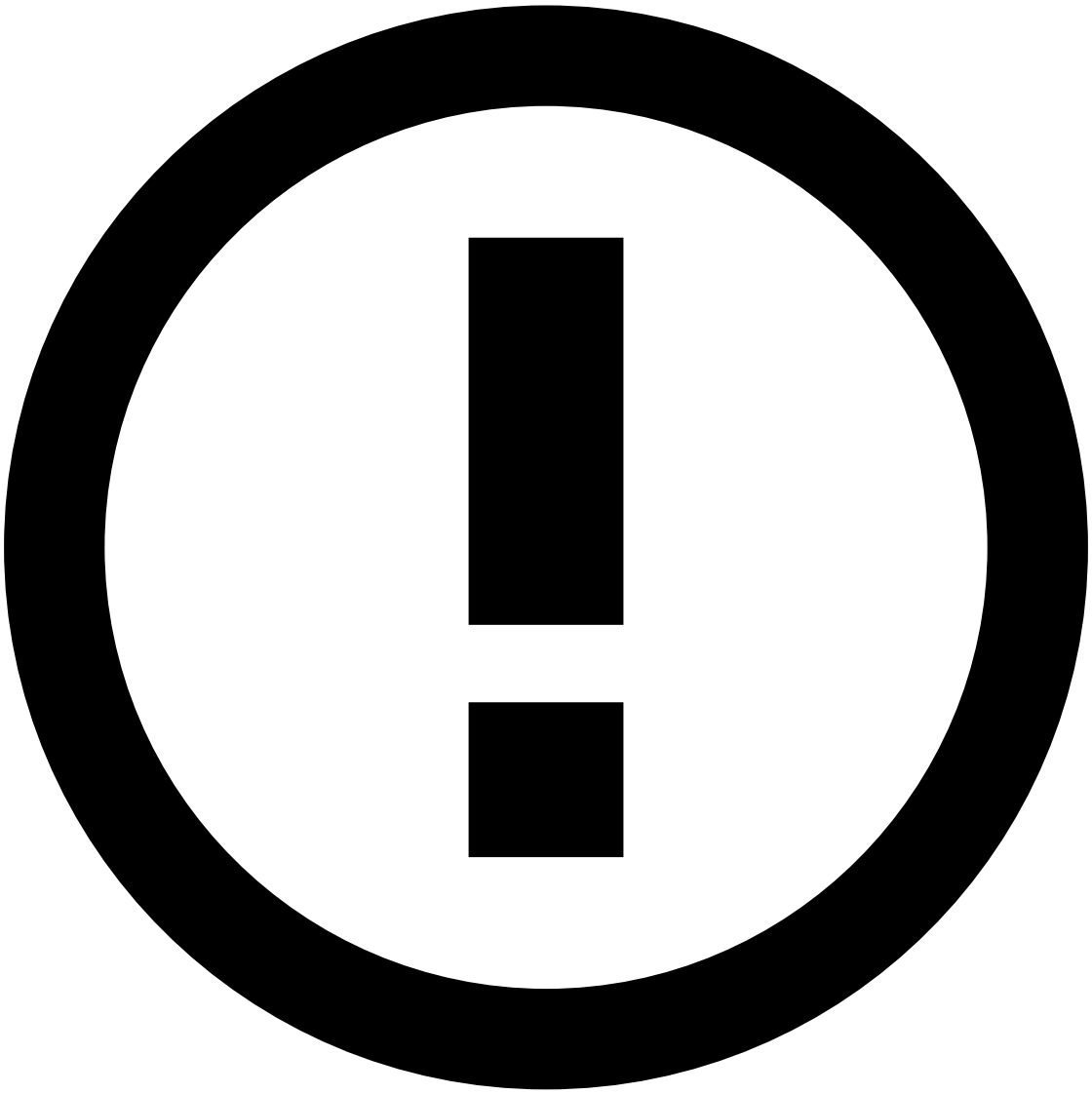
Disputeâ□□

An account holder can contact a bank to file a complaint for different reasons (e.g. an unauthorized payment). In this case, the bank and the account holder may initiate a dispute (e.g. reversal, recall, and refund). The Info endpoint with subtype `dispute` can be used to inform Feedzai about the details of this process. If the process results in a refund, you can use the Feedback endpoint to inform Feedzai about refund-related reasons.

The following diagram shows the end-to-end process:

The steps depicted in the previous image are the following:

1. The sender sends the transfer instructions to the sender bank.
2. The sender bank sends an API request via the **Transfer Initiation** endpoint.
3. The sender bank sends the transfer instructions to the receiver bank.
4. The receiver's account is updated with the funds.
5. The sender contacts the sender bank to initiate a dispute.
6. The sender bank sends an API request via the **Info** endpoint with subtype `dispute` to inform Feedzai about this process.



info

Transfer's confirmation: Once the dispute is resolved, the final information needs to be sent via the Feedback endpoint.

Learn moreâ□□

Check the Settlement Endpoint page to continue learning about the endpoints' life cycles.